



Advanced Data Mining (D7043E)

Assignment 1

Group:

Rapidminer

Group members:

Morteza Salehi

Avijit Saha

Isa Jonsson

Abdalkhman Dabor

Amita Bhuddi

Submitted on:

Task 1 - 2025-10-06

Task 2 - 2025-11-05

Autumn semester

Table of contents

Task 1 - CRISP-DM	3
Business understanding	3
1. Objective	3
2. Project plan	3
3. Success Criteria	3
Data understanding	4
1. Initial Data Collection Report	4
Data Sources	4
Data Acquisition Method	4
Potential Problems	4
2. Describe Data	4
Format	4
Quantity	4
Surface Features	5
Evaluation Against Requirements	5
Potential Issues	5
Recommendations	5
3. Explore Data	5
Outliers	9
4. Verify Data Quality	10
5. Data Quality Report	11
Data Preparation	11
1. Select data	11
2. Clean Data	11
3. Construct Required Data	14
4. Integrate Data	16
5. Visualization	16
3-in-1 component PCA	17
1-per-gesture PCA	20
Task 2 - Data mining	23
1. Models Training	23
i) kNN	23
ii) Decision Tree	25
iii) Random Forest	27
iv) AutoMLP	28
v) SVM	29
2. Results	31
3. Results with PCA	33
4. Conclusion	34

Task 1 - CRISP-DM

Business understanding

1. Objective

The goal of this project is to train and evaluate classifiers to automatically recognize 30 sign-language gestures from Kinect skeleton-derived feature vectors, compare classifier performance, and investigate the effect of dimensionality reduction (PCA at 90% and 80% variance) on the top two models. The main business objective is to show how data mining methods can be applied to the real-world problem of gesture recognition, something that is very relevant in accessibility technologies and sign language interpretation. The main output of the project is going to be a comparative analysis of several classifiers, including k-NN, Random Forest, Decision Tree, SVM, and MLP, with an evaluation of their predictive performance on unseen gesture data.

The team will be using Rapidminer as the software tool for the data mining and the train-final.csv file as the data input for training our models. The models will then be evaluated, with the top two performing ones being used on the test-final.csv data, with the reduction of dimensions using PCA.

2. Project plan

- **Stage 1 (Week 1-2):** Starting up and doing the CRISP-DM analysis of the project, setting up communication (group chats, shared documents), assigning roles, and scheduling recurring meetings based on team members' availability.
- **Stage 2 (Week 3):** Data preprocessing with detection and handling of missing values, and making sure the attribute values are in the appropriate form to be given as input for an algorithm. Visualize the data with different techniques and do a descriptive analysis.
- **Stage 3 (Week 4-5):** Train classifiers on the data (Decision Tree, Random Forest, k-NN, MLP, SVM) and evaluate the classifiers based on accuracy, confusion matrix, and precision/recall.
- **Stage 4 (Week 6):** For the two best classifiers, reduce dimensions by 90 and then by 80 percent using PCA, and evaluate and reflect on the results.

3. Success Criteria

The data mining success criteria will be seen when determining the two best classifiers. The project plan includes the use of a performance operator in RapidMiner to analyze the performance of the classifiers based on accuracy, confusion matrix, and precision/recall (F1-measure).

The business success criteria of the project will be measured by our ability to produce repeatable and interpretable results that highlight strengths and limitations of the classifiers in order to provide insight into which methods are the most effective for this type of high-dimensional motion data.

Data understanding

1. Initial Data Collection Report

Data Sources

- A. train-final.csv: training dataset of gesture instances (240 features + labels).
- B. test-final.csv: test dataset of gesture instances (240 features + labels).

Data Acquisition Method

- A. Provided as CSV files by project resources. The CSV files have a dataset of human gestures recorded using Microsoft Kinect. Kinect captures the 3D positions of 20 body joints as a subject performs a gesture (such as hello, yes, rain). Since each joint has 3 coordinates (x, y, z), that's 60 features per frame. Additionally, cosine angles are calculated, adding another 60 features. So every frame (of the sequence) has 120 features. To make things simpler, instead of keeping the whole sequence (over a time period), the dataset has been summarized by taking the mean and standard deviation of each feature and frame (for one gesture). That gives 240 numbers per gesture, i.e. $120 \text{ features} \times 2 \text{ (mean + standard deviation)} = 240 \text{ numbers per gesture}$.
- B. For exploration, the training dataset has been loaded into RapidMiner.

Potential Problems

- A. A large number of features (240 per row) ensures column parsing during processing.
- B. Gesture labels appear in two formats (string label + numeric ID), need to be standardised before training.
- C. Candidate (string) (subject information) not present, may or may not be used in training. Risk of data leakage if the performer's identity is correlated with the gesture.

2. Describe Data

Format

- A. File Type: CSV (Comma-Separated Values)
- B. Structure: Each row contains: 240 numeric features (representing a sign gesture), 1 gesture name (string), 1 gesture ID (numeric)
- C. Header: No header row present

Quantity

- A. Number of Fields per Row: 242 [240 features + 2 labels(1 gesture name + 1 gesture ID)]
- B. Number of Records: The training and test datasets provided both have 540 rows.

Surface Features

- A. Data Consistency: Most rows have 242 fields, but some rows have missing values (empty fields), in the feature columns.
- B. Feature Values: Numeric features include both positive and negative floating-point numbers, with some scientific notation (e.g., 5.10E-05).
- C. Label Distribution: Gesture names and IDs are categorical and appear at the end of each row.
- D. No explicit missing value markers (e.g., "NA" or "null"), but empty fields are present in some rows.

Evaluation Against Requirements

- A. Includes both a categorical label (gesture name) and a numeric ID for supervised learning.

Potential Issues

- A. No header row.
- B. Some rows have missing feature values (empty fields) so, empty fields may need to be handled during preprocessing.
- C. The file is large, so efficient processing is important.

Recommendations

- A. Add a header row for clarity and easier processing.
- B. Check for and handle missing values in the feature columns.
- C. Validate row consistency (ensure all rows have 242 fields).
- D. No need to keep both gesture ID and gesture name columns; either one is sufficient.

3. Explore Data

A total of 30 labels (gestures) are present in the file:

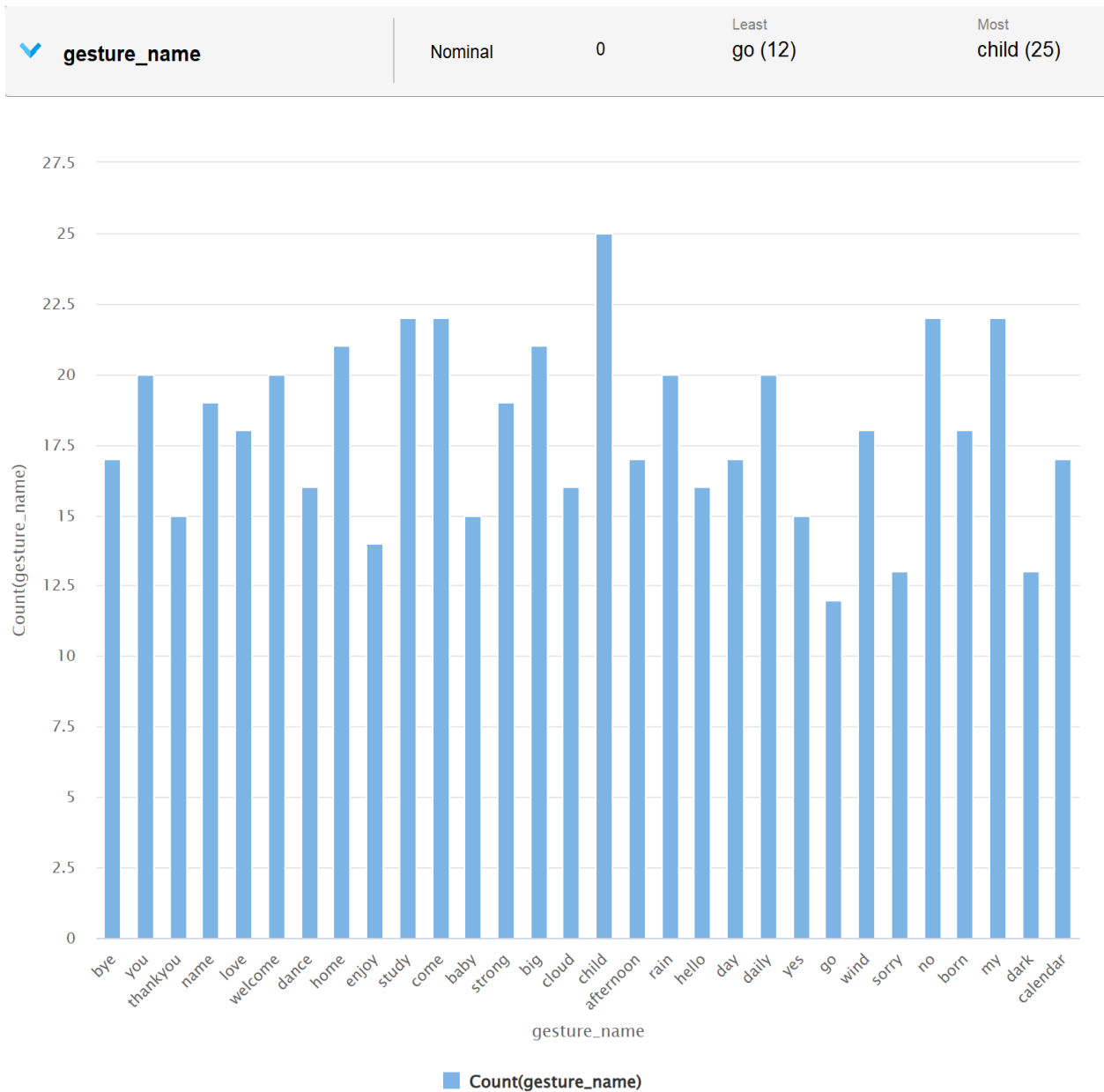
gesture_id	gesture_name
1	afternoon
2	baby
3	big
4	born
5	bye
6	calendar
7	child

gesture_id	gesture_name
8	cloud
9	come
10	daily
11	dance
12	dark
13	day
14	enjoy
15	go
16	hello
17	home
18	love
19	my
20	name
21	no
22	rain
23	sorry
24	strong
25	study
26	thankyou
27	welcome
28	wind
29	yes
30	you

The top 3 gestures in the data are:

- 1. Child: 25 counts
- 2. No, my, come, study: 22 counts
- 3. Big: 21 counts

The least prevalent gesture is “go”, which appears just 12 times.



7 data rows have missing attributes. These are missing for 1 row of each of the following gestures:

- 1. Baby
- 2. Strong

3. Big
4. Cloud
5. Welcome
6. Name
7. No

Row No.	gestur...	gestur...	att1	att2	att3	att4	att5	att6	att7	att8	att9	att10
1	baby	2	0.204	0.619	2.225	0.226	0.426	2.265	0.051	?	?	?
2	strong	24	-0.148	0.695	2.387	-0.172	0.515	2.449	-0.290	?	?	?
3	big	3	0.172	0.659	2.448	0.187	0.463	2.470	0.016	?	?	?
4	cloud	8	-0.366	0.557	2.435	-0.378	0.373	2.473	-0.543	?	?	?

Row No.	gestur...	gestur...	att1	att2	att3	att4	att15	att16	att17	
1	baby	2	0.204	0.619	2.225	0.226	2.260	0.446	0.137	
2	strong	24	-0.148	0.695	2.387	-0.172	2.240	0.215	0.437	
3	big	3	0.172	0.659	2.448	0.187	2.343	0.453	0.207	
4	cloud	8	-0.366	0.557	2.435	-0.378	2.326	-0.036	0.302	
5	welcome	27	-0.088	0.706	2.424	-0.118	?	?	?	
6	name	20	0.045	0.605	2.115	0.001	?	?	?	
7	no	21	-0.046	0.704	2.414	-0.088	?	?	?	

The following attributes (Features) share positive covariances:

1. Att1
2. Att4
3. Att 7
4. Att13
5. Att19
6. Att22
7. Att25
8. Att28
9. Att31
10. Att34
11. Att37
12. Att40
13. Att43
14. Att46
15. Att49
16. Att52
17. Att55
18. Att58

Attribu...	att1	att2	att3	att4	att5	att6	att7	att8	att9	att10	att11	att12
att1	0.052	0.003	-0.009	0.053	0.003	-0.010	0.052	?	?	?	0.004	-0.010
att2	0.003	0.003	0.001	0.004	0.003	0.001	0.003	?	?	?	0.004	0.002
att3	-0.009	0.001	0.010	-0.009	0.001	0.010	-0.010	?	?	?	0.000	0.008
att4	0.053	0.004	-0.009	0.054	0.003	-0.010	0.053	?	?	?	0.004	-0.011
att5	0.003	0.003	0.001	0.003	0.004	0.001	0.003	?	?	?	0.004	0.002
att6	-0.010	0.001	0.010	-0.010	0.001	0.011	-0.011	?	?	?	0.000	0.008
att7	0.052	0.003	-0.010	0.053	0.003	-0.011	0.053	?	?	?	0.004	-0.009
att8	?	?	?	?	?	?	?	?	?	?	?	?
att9	?	?	?	?	?	?	?	?	?	?	?	?
att10	?	?	?	?	?	?	?	?	?	?	?	?
att11	0.004	0.004	0.000	0.004	0.004	0.000	0.004	?	?	?	0.005	0.002
att12	-0.010	0.002	0.008	-0.011	0.002	0.008	-0.009	?	?	?	0.002	0.011
att13	0.049	0.004	-0.010	0.050	0.004	-0.012	0.052	?	?	?	0.005	-0.008
att14	0.004	0.003	0.002	0.004	0.003	0.002	0.004	?	?	?	0.004	0.002
att15	?	?	?	?	?	?	?	?	?	?	?	?
att16	?	?	?	?	?	?	?	?	?	?	?	?
att17	?	?	?	?	?	?	?	?	?	?	?	?
att18	-0.008	0.002	0.006	-0.011	0.002	0.006	-0.007	?	?	?	0.003	0.014
att19	0.048	0.004	-0.010	0.048	0.004	-0.013	0.053	?	?	?	0.006	-0.006
att20	0.003	0.003	0.003	0.004	0.004	0.003	0.004	?	?	?	0.005	0.003

Outliers

Outliers as per detect outlier (distance method).

For the distance formula, Euclidean distance, Squared distance:

Open in
 Turbo Prep
 Auto Model
 Interactive Analysis
Filter (10 / 10 examples): all ▼

Row No.	id	gesture_na...	gesture_id	outlier	att1	att2	att3	att4	att5	
1	37	cloud	8	true	-0.294	0.680	2.471	-0.286	0.494	
2	73	wind	28	true	-0.327	0.674	2.449	-0.325	0.485	
3	110	daily	10	true	-0.091	0.637	2.345	-0.095	0.445	
4	178	daily	10	true	-0.076	0.635	2.338	-0.086	0.443	
5	195	daily	10	true	-0.086	0.637	2.346	-0.088	0.446	
6	283	no	21	true	0.127	0.622	2.188	0.091	0.436	
7	285	come	9	true	0.134	0.606	2.092	0.108	0.419	
8	378	cloud	8	true	0.009	0.690	2.333	-0.064	0.516	
9	479	come	9	true	0.148	0.596	2.061	0.121	0.411	
10	511	yes	29	true	0.108	0.584	2.003	0.100	0.398	

For the distance formula, cosine distance:

Row No.	Id	gesture_na...	gesture_id	outlier	att1	att2	att3	att4	att5
1	181	no	21	true	-0.142	0.701	2.405	-0.140	0.513
2	243	come	9	true	-0.209	0.721	2.404	-0.220	0.539
3	316	go	15	true	-0.142	0.664	2.310	-0.198	0.471
4	338	day	13	true	-0.271	0.711	2.242	-0.307	0.527
5	340	calendar	6	true	-0.343	0.663	2.386	-0.362	0.475
6	414	study	25	true	-0.295	0.674	2.412	-0.337	0.490
7	464	go	15	true	-0.139	0.660	2.299	-0.193	0.473
8	496	day	13	true	-0.282	0.709	2.239	-0.308	0.523
9	503	study	25	true	-0.307	0.669	2.410	-0.329	0.484
10	535	born	4	true	-0.295	0.671	2.404	-0.345	0.487

For the distance formula, inverted cosine distance and angle:

Row No.	Id	gesture_na...	gesture_id	outlier	att1	att2	att3	att4	att5
1	37	cloud	8	true	-0.294	0.680	2.471	-0.286	0.494
2	110	daily	10	true	-0.091	0.637	2.345	-0.095	0.445
3	128	my	19	true	0.116	0.628	2.286	0.117	0.433
4	195	daily	10	true	-0.086	0.637	2.346	-0.088	0.446
5	283	no	21	true	0.127	0.622	2.188	0.091	0.436
6	285	come	9	true	0.134	0.606	2.092	0.108	0.419
7	378	cloud	8	true	0.009	0.690	2.333	-0.064	0.516
8	428	yes	29	true	0.103	0.580	1.985	0.091	0.393
9	479	come	9	true	0.148	0.596	2.061	0.121	0.411
10	511	yes	29	true	0.108	0.584	2.003	0.100	0.398

Outliers as per LOF outlier detector (LOF value > 1.5) :

Row No.	Id	gesture_na...	outlier	att1	att2	att3	att4	att5	att6
1	50	wind	1.596	1.290	-0.322	-1.265	1.400	-0.414	-1.589
2	73	wind	1.660	-0.657	0.533	1.191	-0.560	0.459	1.500
3	164	welcome	1.512	-1.101	-0.242	-1.454	-0.923	0.135	-0.554
4	285	come	1.564	1.363	-0.608	-2.436	1.300	-0.665	-2.615
5	481	enjoy	1.741	-1.537	-1.631	0.564	-1.660	-1.706	0.449
6	527	strong	1.701	0.908	-0.840	-2.718	1.045	-0.440	-2.702
7	540	welcome	1.520	0.009	-0.179	-2.223	-0.264	0.039	-1.602

4. Verify Data Quality

Data Completeness: All gestures are represented in the data. Total being 30 gestures.

Missing Values: There are 7 data rows out of a total of 540 that have missing attributes. Values are missing in att8, att9, att10 (for 4 rows) and att15, att16 and att17 (for 3 rows).

5. Data Quality Report

In general, the data seems to be of good quality as all gestures are represented well in the data with sufficient features. The number of missing features is also low and these are not skewed towards one gesture.

Data Preparation

1. Select data

In this phase, we included the *train-final.csv* dataset for human gesture analysis recorded using Microsoft Kinect. The dataset comprises 540 rows and 242 attributes in total: 240 numeric features representing each sign gesture, plus 2 label attributes (gesture_name as a string and gesture_id as a numeric value).

Row No.	att1	att2	att3	att4	att5	att6	att7	att8
1	-0.480	0.726	2.404	-0.482	0.541	2.456	-0.646	0.406
2	0.052	0.647	2.410	0.060	0.452	2.449	-0.107	0.344
3	-0.003	0.625	2.240	0.019	0.436	2.270	-0.153	0.336
4	-0.029	0.664	2.443	-0.023	0.481	2.452	-0.203	0.353
5	0.066	0.639	2.378	0.069	0.447	2.406	-0.093	0.339
6	-0.470	0.532	2.414	-0.527	0.352	2.493	-0.669	0.247
7	0.027	0.646	2.303	0.037	0.468	2.336	-0.149	0.390
8	0.056	0.623	2.214	0.060	0.438	2.255	-0.106	0.323
9	-0.421	0.712	2.327	-0.419	0.525	2.390	-0.598	0.411
10	-0.505	0.565	2.464	-0.529	0.387	2.484	-0.689	0.247
11	-0.406	0.522	2.284	-0.425	0.348	2.338	-0.580	0.290

Figure 1: Statistics View - Statistics tab showing 540 rows and 242 attributes.

2. Clean Data

In this phase, we addressed missing values and errors to enhance the quality of the training set (*train-final.csv*) in accordance with the CRISP-DM methodology. After retrieving the dataset, we found that there are missing values in seven rows across specific attributes. These were addressed using the Replace Missing Values operator with the mean imputation method. Since the number of missing values was relatively small compared to the 540 rows, the impact on the dataset statistics was considered minimal.

Data

Statistics

Visualizations

Annotations

Open in

Turbo Prep

Auto Model

Interactive Analysis

Filter (7 / 540 examples):

missing_attributes

Row No.	outlier	att8	att9	att15	att16	att1	att2	att3	att4	att5	att6
1	0	?	?	2.260	0.446	0.204	0.619	2.225	0.226	0.426	2.265
2	0	?	?	2.240	0.215	-0.148	0.695	2.387	-0.172	0.515	2.449
3	0	?	?	2.343	0.453	0.172	0.659	2.448	0.187	0.463	2.470
4	0	?	?	2.326	-0.036	-0.366	0.557	2.435	-0.378	0.373	2.473
5	0	0.380	2.374	?	?	-0.088	0.706	2.424	-0.118	0.514	2.456
6	0	0.281	2.060	?	?	0.045	0.605	2.115	0.001	0.419	2.144
7	0	0.374	2.342	?	?	-0.046	0.704	2.414	-0.088	0.514	2.442

Figure 2: Table view showing the missing values in (train-final.csv)

Process

Process

90%

Retrieve train-final

Replace Missing Values

Recommended Operators

Parameters

Replace Missing Values

attribute filter type

all

invert selection

include special attributes

default

average

[Show advanced parameters](#)

[Change compatibility \(11.1.001\)](#)

Figure 3: Replace Missing Values Process - Screenshot of mean replacement settings.

Row No.	outlier	att1	att2	att3	att4	att5	att6	att7	att8	att9	att10
---------	---------	------	------	------	------	------	------	------	------	------	-------

Figure 4: Cleaned Data Table - Table view confirming no missing values in (train-final.csv).

After replacing missing values, we checked for potential outliers in the data. Therefore, we utilised the Detect Outlier (Distances) operator, as it provides clear results for interpretation and reporting. This method measures the Euclidean distance of each instance to its nearest neighbours. We kept the default settings in RapidMiner, with both the lower and upper bounds set to 10. As a result, the operator flagged 10 instances as outliers. Subsequently, we added the Filter Examples process to exclude these 10 outliers from the dataset.

Row No.	outlier	att1	att2	att3	att4	att5	att6	att7	att8	att9
43	true	-0.476	0.558	2.446	-0.495	0.370	2.503	-0.651	0.260	2.463
71	true	-0.432	0.535	2.319	-0.443	0.344	2.372	-0.612	0.239	2.374
254	true	-0.427	0.534	2.316	-0.441	0.343	2.367	-0.610	0.239	2.374
285	true	0.134	0.606	2.092	0.108	0.419	2.102	0.027	0.269	2.003
293	true	0.121	0.585	2.030	0.103	0.407	2.062	0.040	0.268	1.959
432	true	0.117	0.625	2.159	0.108	0.448	2.167	0.037	0.298	2.065
434	true	0.145	0.626	2.245	0.114	0.441	2.287	0.013	0.302	2.199
476	true	-0.484	0.542	2.350	-0.524	0.357	2.380	-0.631	0.227	2.275
481	true	-0.529	0.546	2.387	-0.582	0.357	2.417	-0.661	0.235	2.305
492	true	0.080	0.613	2.144	0.059	0.430	2.166	-0.075	0.299	2.102
1	false	-0.480	0.726	2.404	-0.482	0.541	2.456	-0.646	0.406	2.428
2	false	0.052	0.647	2.410	0.060	0.452	2.449	-0.107	0.344	2.461
3	false	-0.003	0.625	2.240	0.019	0.436	2.270	-0.153	0.336	2.305

Figure 5: Outlier Detection (Distances) Result - Result History showing 10 instances flagged as True (outliers) out of 540 rows.

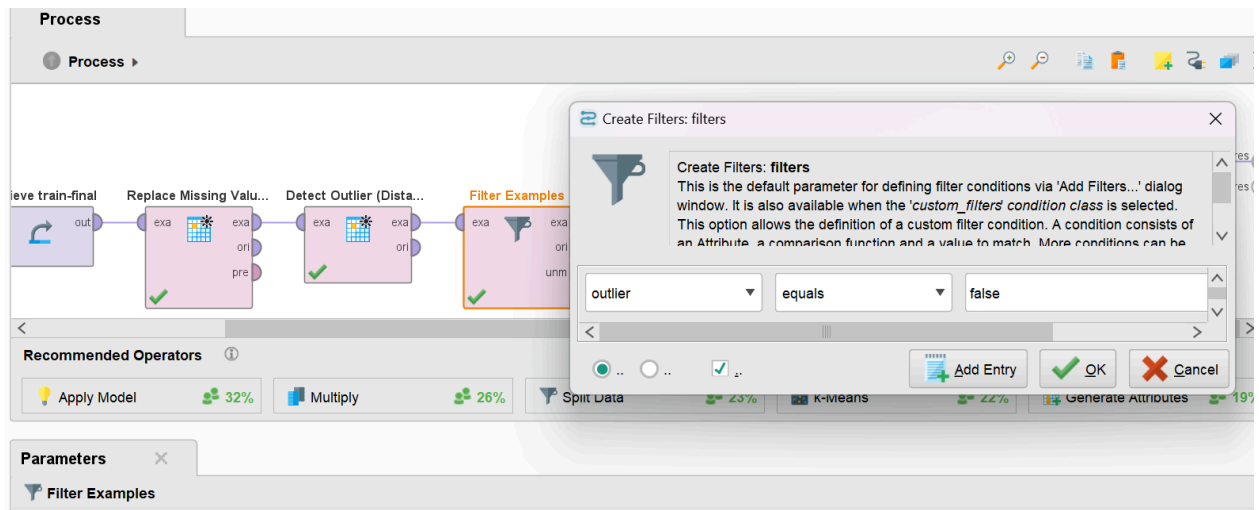


Figure 6: Filter Examples operator configuration in RapidMiner, showing that only instances with an outlier value equal to false are included.

3. Construct Required Data

In this phase, we are going to use the cleaned data from the previous step for modelling per CRISP-DM. We decided to exclude the “gesture_id” column that contains numerical IDs and keep “gesture_name” as the label for classification, since it is more interpretable. We removed the “gesture_id” by using the Select Attributes operator and passing the attribute “att242”, which represents the “gesture_id”.

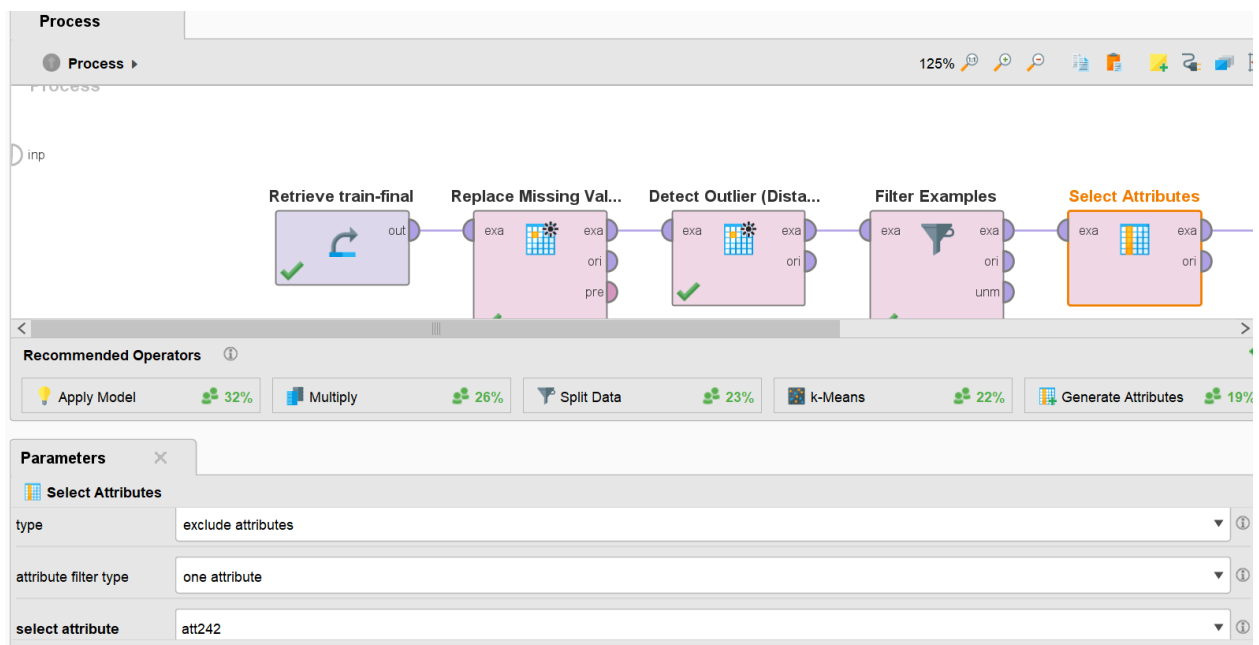


Figure 7: Select Attributes Settings - Screenshot showing exclusion of "gesture_id"

After excluding the "gesture_id", we set a label for classification (The result we are trying to predict), which in our case is the "gesture_name". Therefore, we are using the Set Role process and adding att241(gesture_name) as a label.

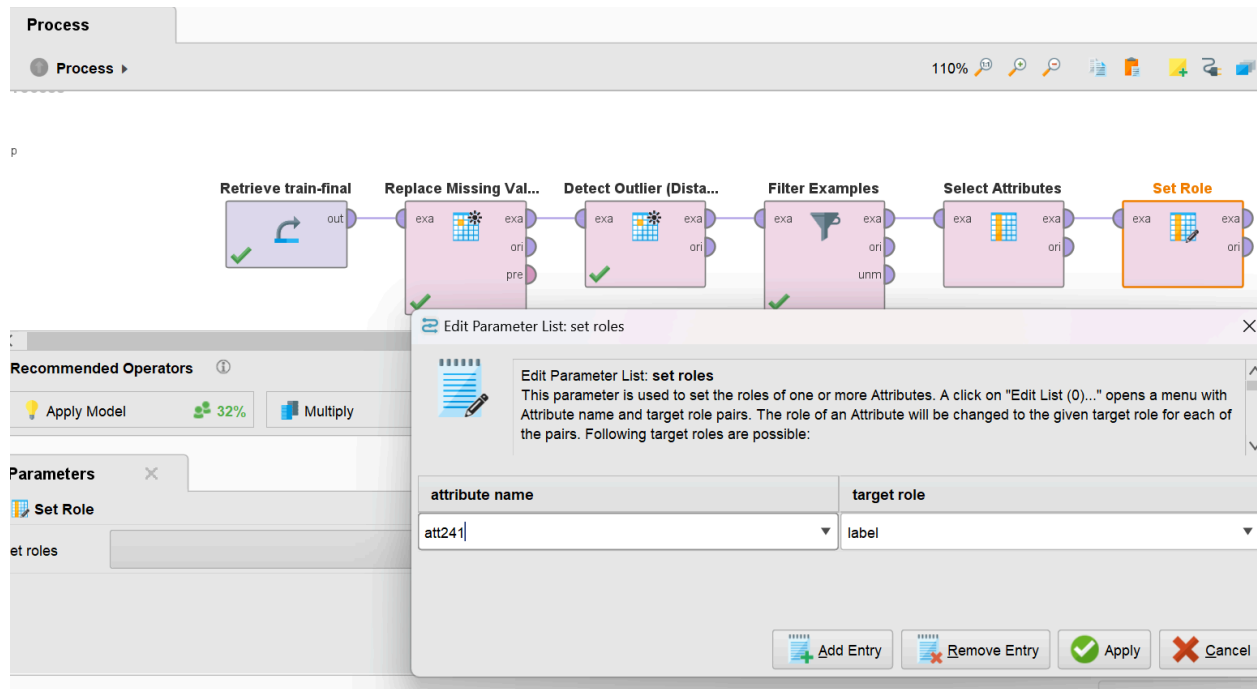


Figure 8: Set Role Configuration - Screenshot of att241(gesture_name) set as label.

After excluding the "gesture_id" with the 10 outliers and including "gesture_name" as a label, we applied the Normalization operator to rescale all feature values to a comparable range. Normalization is applied to all features in the dataset, with the Z-transformation method, since it standardizes all features to have a mean of 0 and a standard deviation of 1, which is beneficial for distance-based classifiers (k-NN, SVM).

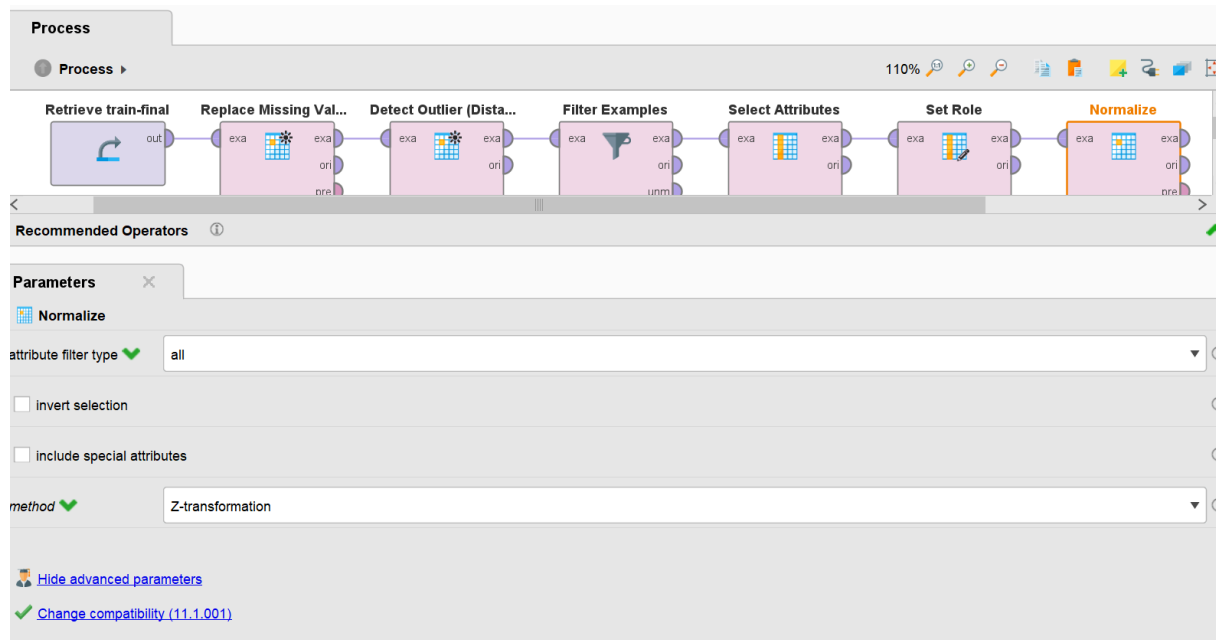


Figure 9: Normalize operator set to Z-transformation.

4. Integrate Data

Both test-final.csv and train-final.csv sets were provided in the same structure. No integration across multiple datasets was required. We also ensured that both sets remain separate to prevent data leakage.

5. Visualization

We applied Principal Component Analysis (PCA) with three dimensions to reduce the 240 features of the gesture dataset and then visualized the results in a 3D scatter plot in RapidMiner. This allowed us to see how the different gestures are distributed.

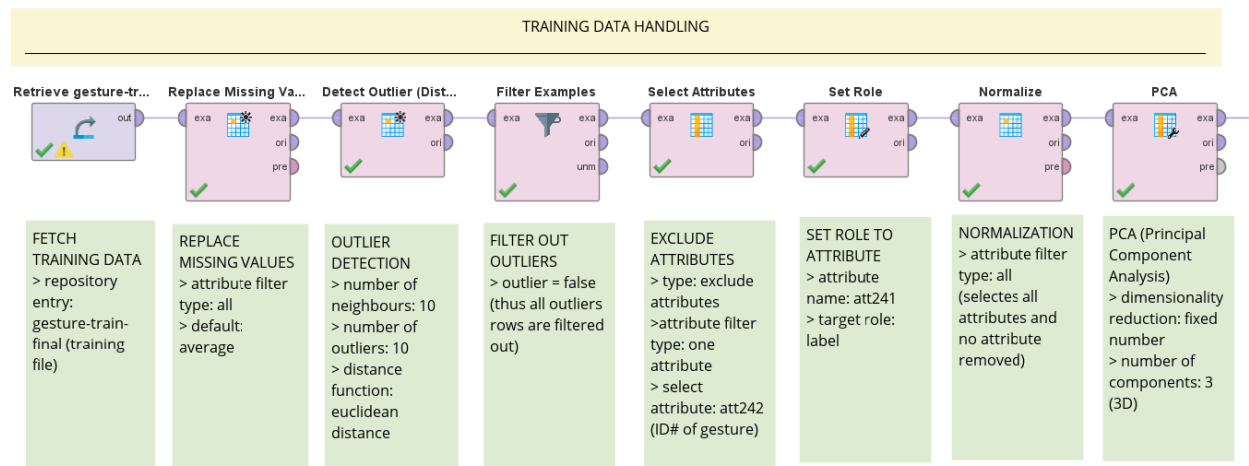


Figure 10: Whole flow including PCA

3-in-1 component PCA

We noticed that some gestures, like *Wind* and *Welcome*, had points that were more spread out. This means that these gestures are performed with more variation between different people or repetitions. Other gestures, like *Dance* and *Home*, had points that were closer together, which means they are performed more similarly and have less variation.

From this, we learned that some gestures may be easier for classification because they form tighter clusters, while others are harder since they show a wider spread and less clear boundaries. The PCA visualization gave us a good overview of how consistent the different gestures are in the dataset.

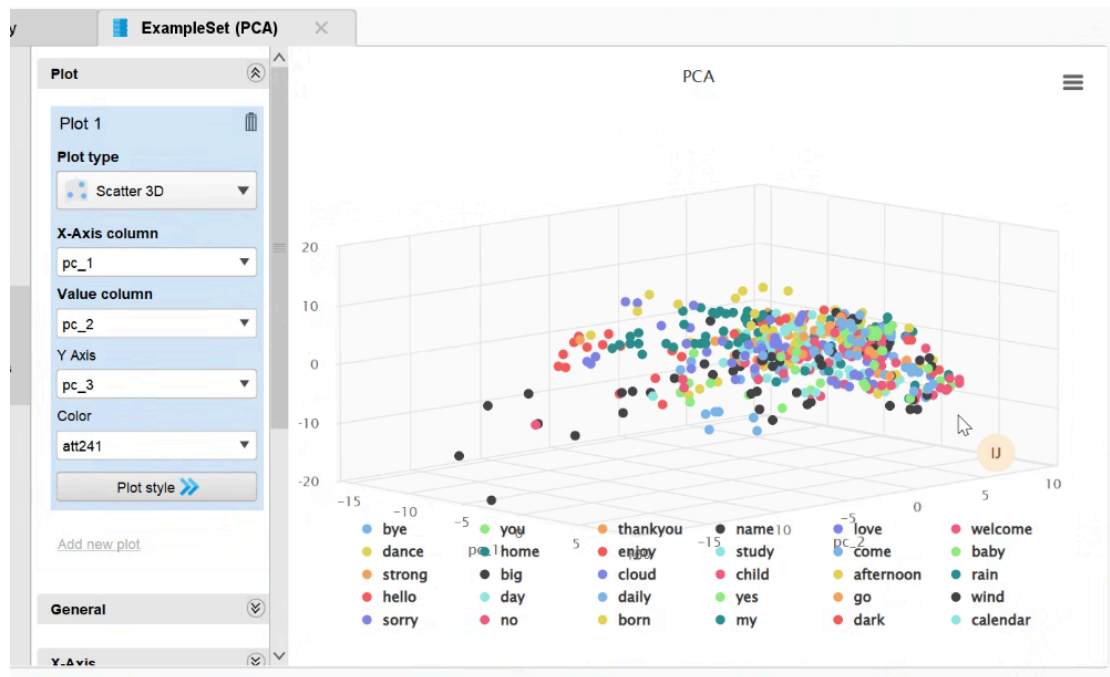


Figure 10: The 3 dimensions of PCA (All gestures)

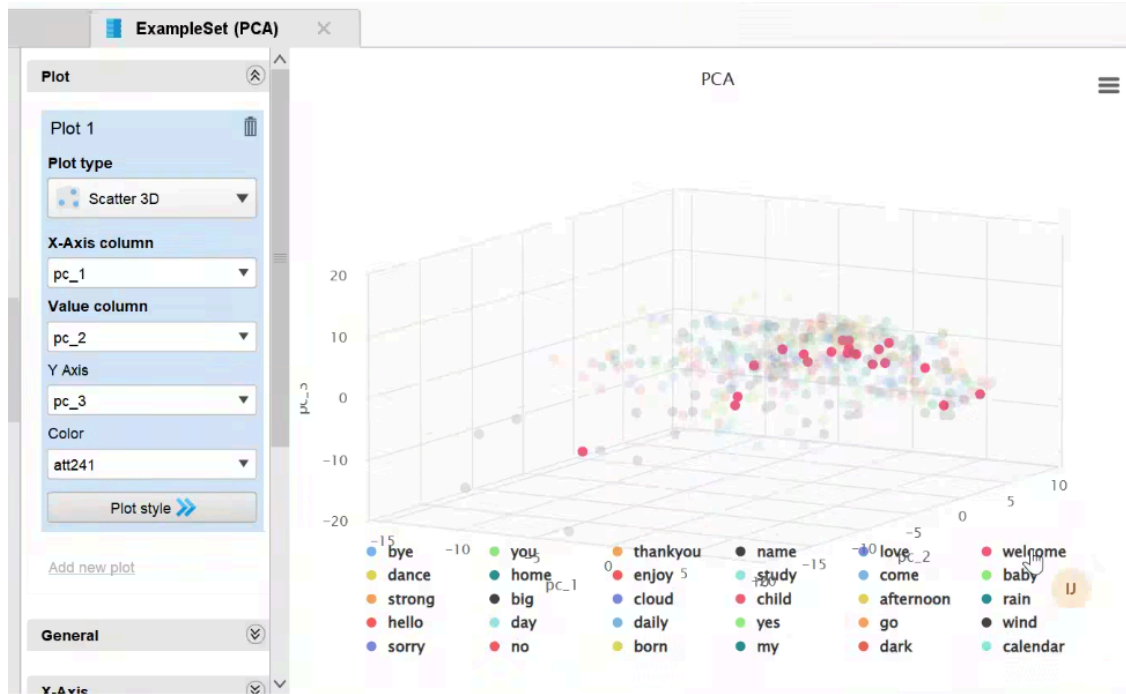


Figure 11: The plots for the gesture 'Welcome'

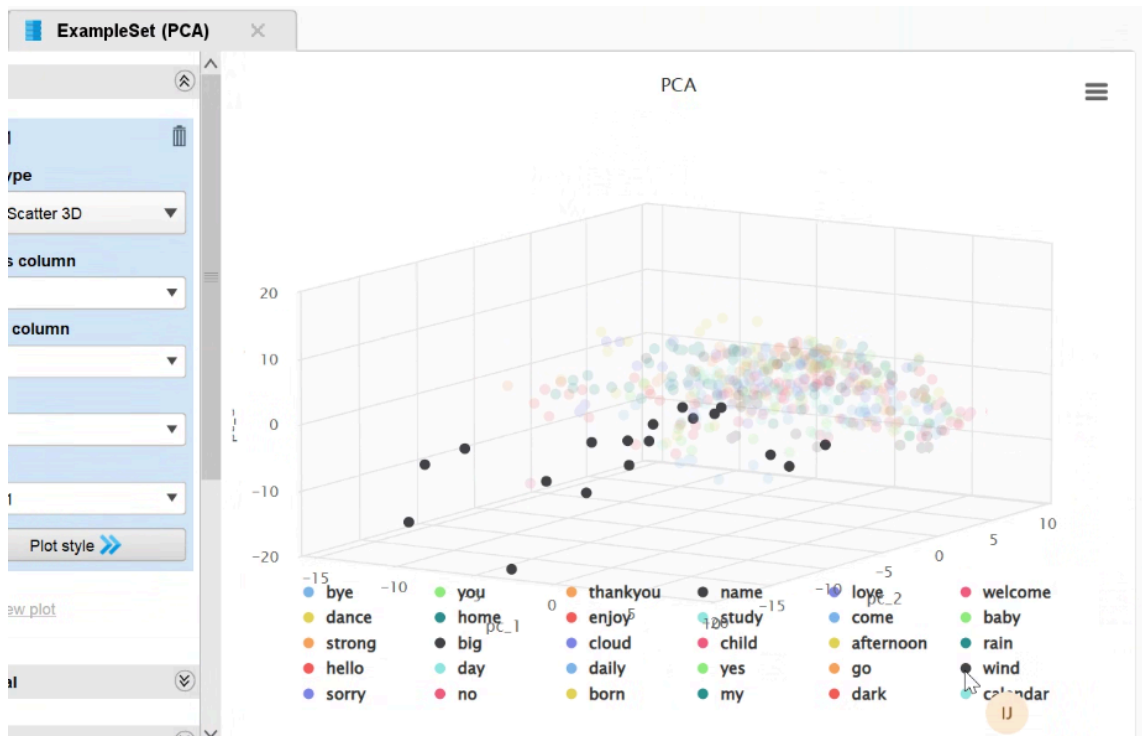


Figure 12: The plots for the gesture 'Wind'

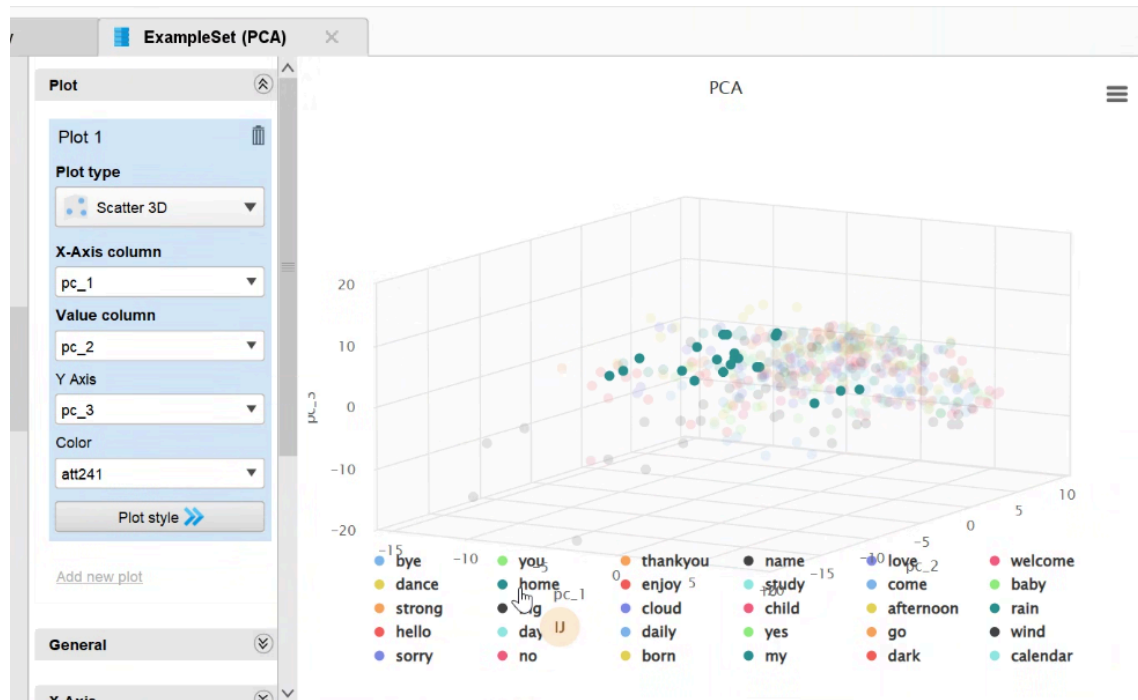


Figure 13: The plots for the gesture 'Home'

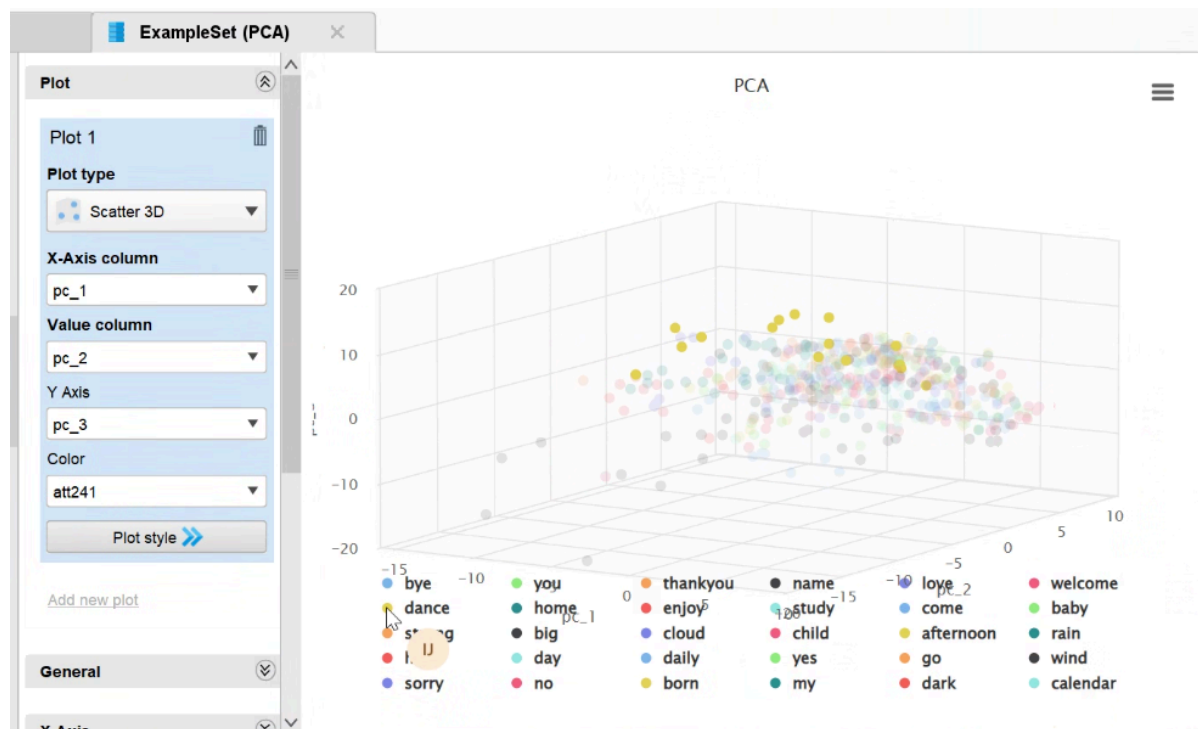


Figure 14: The plots for the gesture 'Dance'

1-per-gesture PCA

To visualize gestures for each component and look at their average moves, this was the result. Some clear patterns that are seen from this are:

- 6 gestures (as also shown in the blue box) - including - home, come, baby, strong, rain, yes, wind are more active on the left side (assuming that the positive axis is the right and the negative axis is left) based on the min/max values on each side.
- The trend line also supports it, but with the top 5 gestures more active in the right, based on the average being - home, rain, go, wind, dark.
- Another insight from this graph is also gestures which are very much centred and are symmetrical in the input data are ones close to value 0, including - come, strong, day, yes.

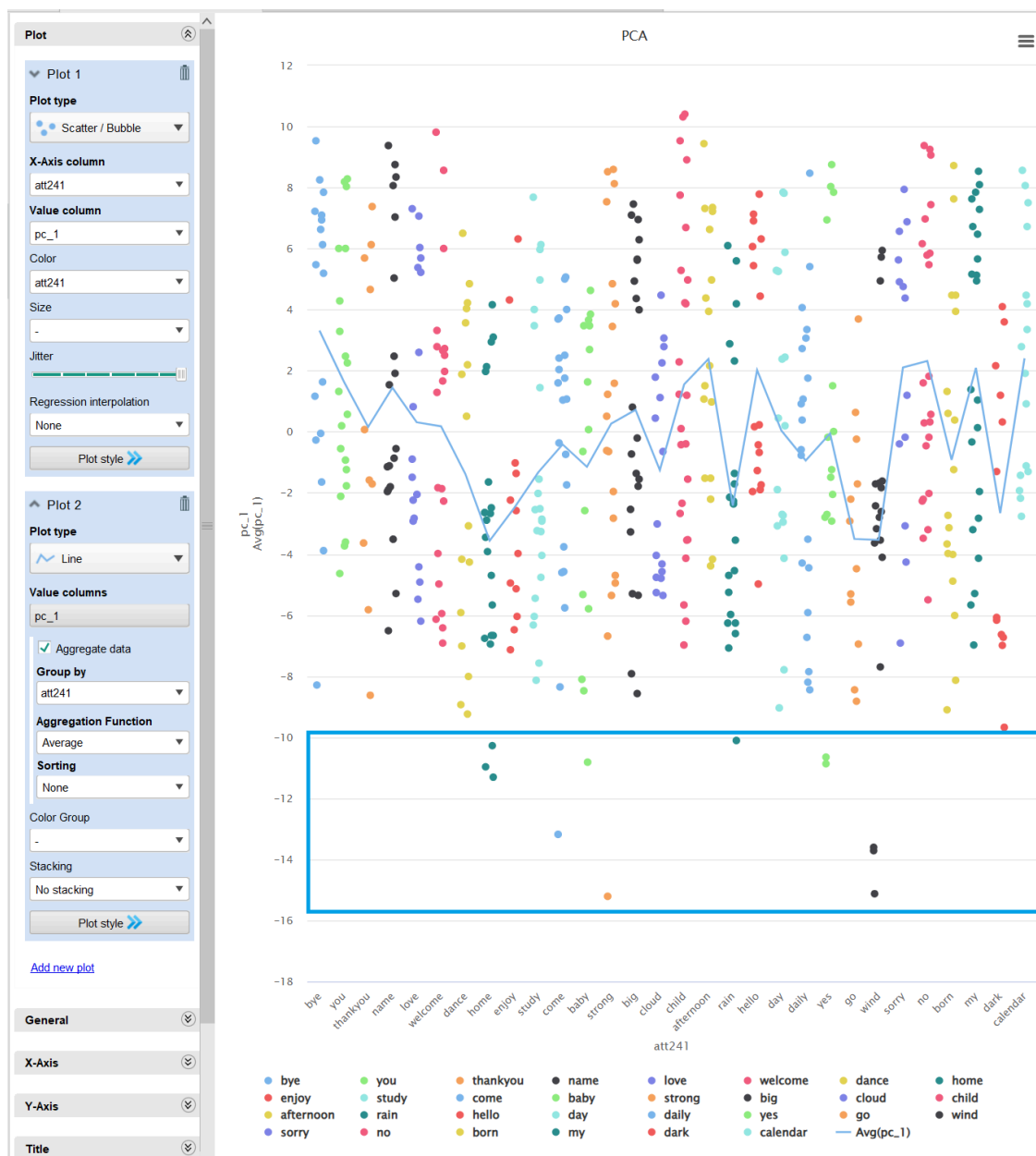


Figure 15: The plots for the gestures and analysis per component, here pc_1, with a trend line based on average values

In the similar way, let's analyze pc_2,

- Very much active on the left for - cloud, wind and dark, looking at the average trend-line and slightly more for - home and rain.
- Not many gestures for this attribute are centred around 0, but some close gestures are - love and calendar.



Figure 16: The plots for the gestures and analysis per component, here pc_2, with a trend line based on average values

Let's also analyze the third value, pc_3,

- Most active towards the left are - name, baby, child, daily and wind based on the average trendline.
- Some more balanced gestures (close to 0) are - you, love, yes, no, born, calendar. Interestingly, not everything is coherent across all three values (graphs)

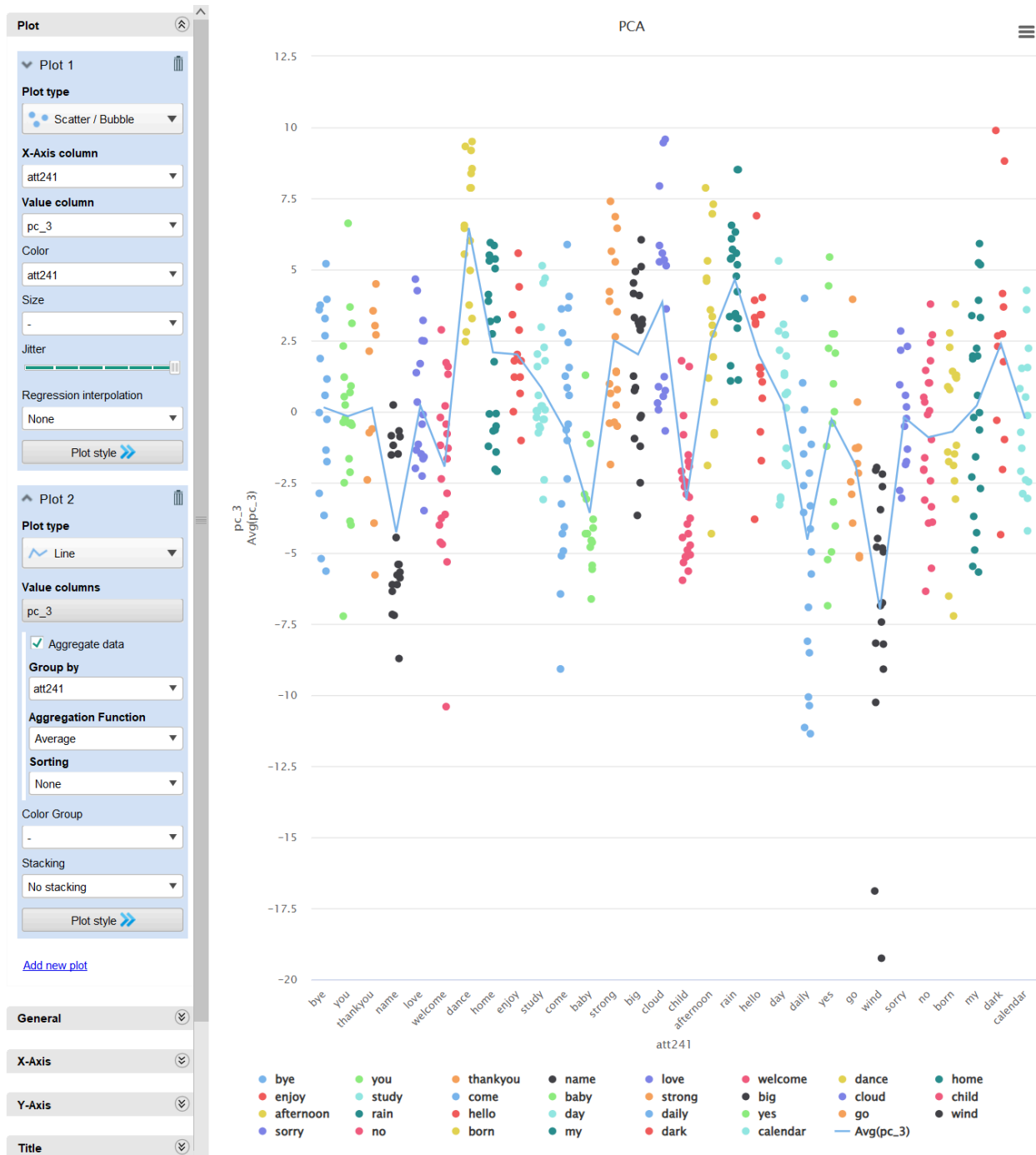
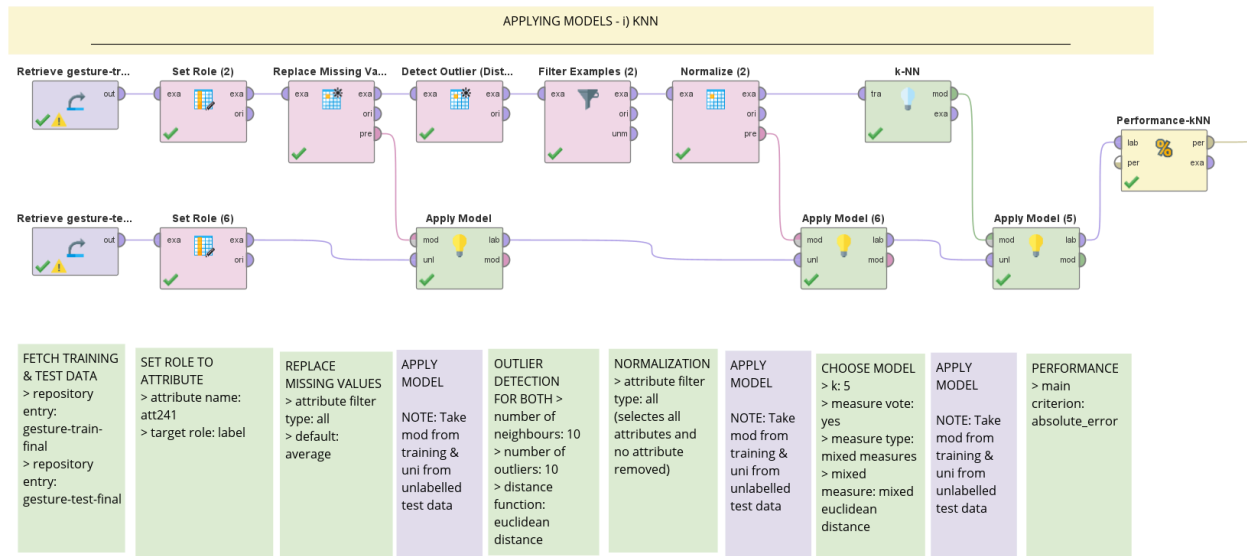


Figure 17: The plots for the gestures and analysis per component, here pc_3, with a trend line based on average values

Task 2 - Data mining

1. Models Training

i) kNN

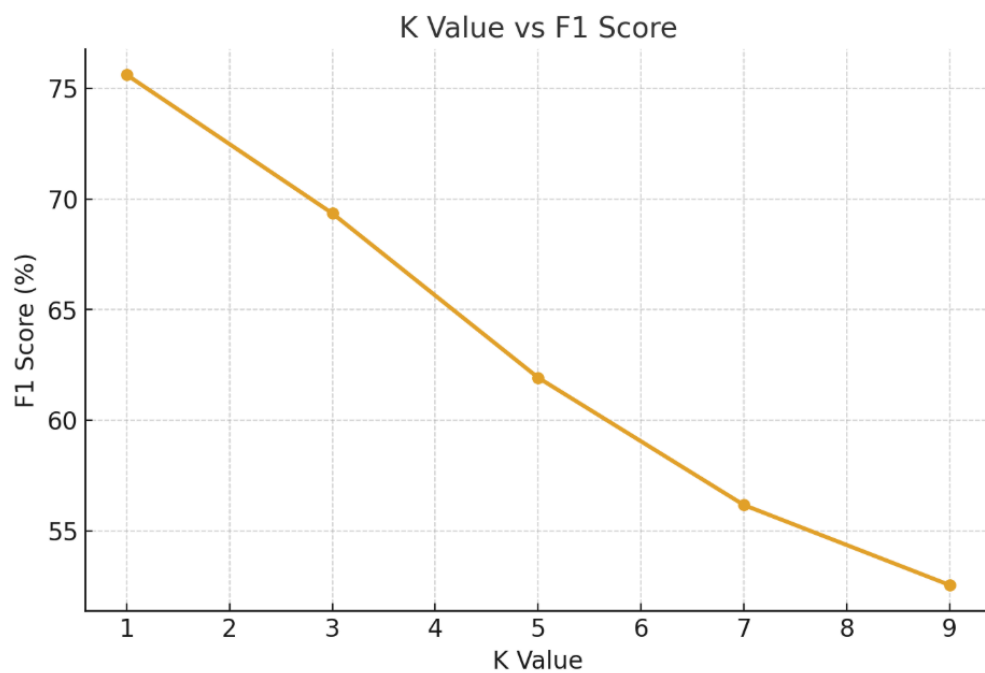


In this flow, the k-Nearest Neighbors (k-NN) model is used as the classification mechanism. The k-NN algorithm is a non-parametric, instance-based learning technique that assigns class labels to new data points by looking at the k closest examples from the training data and deciding the class of a new data point based on which class most of those neighbors belong to. The configuration settings applied in this process are as follows:

- **Voting:** Weighted voting, where the influence of each neighbor is inversely proportional to its distance from the query instance. The adoption of a weighted voting strategy enhances classification precision, particularly near decision boundaries, by giving greater importance to nearer neighbors.
- **Measure type:** Mixed measures. The use of mixed measures enables the model to effectively process datasets containing both numerical and categorical attributes.
- **Distance metric:** Mixed Euclidean distance. The mixed Euclidean distance metric ensures that the similarity computation accounts for variations across different attribute types and scales, thereby improving the robustness of the classification outcome.

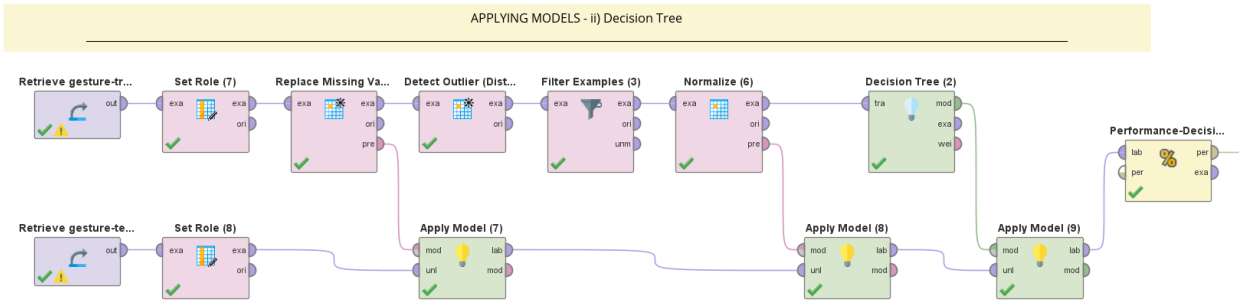
The model exhibited the following behavior for different values of K.

K value	Weighted Precision (%)	Weighted Recall (%)	F1 Score (%)	Likely behaviour of K-NN
1	76.7	74.57	75.62	Memorizes training data and overfitting
3	71.9	66.92	69.35	Some smoothing, better generalization
5	64.82	59.26	61.92	Some smoothing, better generalization
7	58.23	54.24	56.16	Over smoothing, high bias and underfitting
9	56.27	49.29	52.54	Over smoothing, high bias and underfitting



As shown, the F1 score steadily decreases as K increases showing that the model's ability to correctly balance precision and recall weakens when neighborhoods grow too large.

ii) Decision Tree



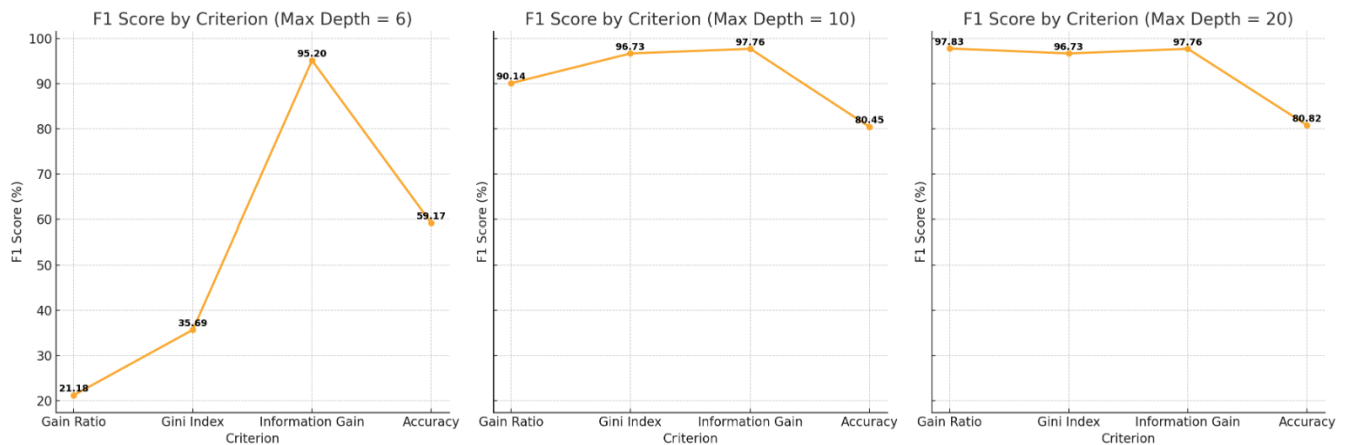
In this flow, the **Decision Tree** operator is used to build a predictive model using the cleaned and prepared training data. A decision tree works by repeatedly dividing the data into smaller groups based on the attribute (or feature) that best separates the target categories. This splitting continues until the model reaches a stopping point, such as a maximum depth or minimum number of examples in a group. The Decision Tree was configured with the following settings:

- Criterion: Gain ratio, Gini Index, Information Gain and Accuracy. N.B: Least squares does not work with nominal labels so was not used to train/evaluate the decision tree model.
 - Gini Index: It measures the impurity of a node. A node is pure (Gini = 0) if all samples belong to one class. The formula penalizes mixed class distributions. The decision tree algorithm selects splits that minimize the Gini index, meaning produce purer child nodes.
 - Information Gain: It measures the reduction in entropy (or uncertainty) after a dataset is split on a feature. It compares entropy before and after a split. The higher the gain, the better the feature separates the classes. The goal is to maximize information gain at each split to make the data as orderly (low entropy) as possible.
 - Gain Ratio: It is a normalized version of Information Gain that penalizes features with many distinct values. It works by dividing Information Gain by the intrinsic information (a measure of how broadly the feature splits the data). It Prevents bias toward attributes with many categories (like IDs).
 - Accuracy: It is a measure of the overall proportion of correctly classified samples. $\text{Accuracy} = (\text{True Positives} + \text{True Negatives}) / \text{Total Samples}$. It can be misleading if the dataset is imbalanced — hence other metrics like precision, recall, and F1 are often also used.
- Maximum Depth: 6, 10 and 20
- Apply Pruning: Yes
Pruning removes parts of the tree that don't help improve accuracy. This makes the model simpler and more reliable.
- Confidence: 0.1
This value controls how aggressive the pruning is. A lower value like 0.1 means the model will

prune more, helping to reduce overfitting.

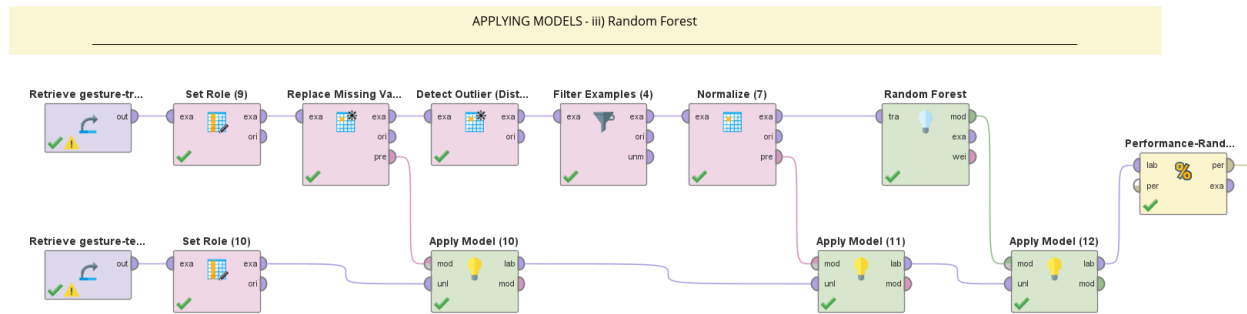
- **Apply Pre-Pruning: Yes**
This stops unnecessary splits from happening during the tree-building process, which helps keep the model efficient and focused.
- **Minimum Gain: 0.01**
The tree will only split if the gain in information is at least 0.01. This prevents the model from making weak or unhelpful splits.
- **Minimum Leaf Size: 2**
Each final group (or leaf) must contain at least two examples. This avoids creating overly specific rules that only apply to a single case.

The decision tree model exhibited the following behaviour at different maximal depths with different criteria.



Maximal Depth	Best Criterion/Criteria	F1 Trend	Comment
6	Information Gain	Sharply dominates others	Others underfit
10	Information Gain (F1 ≈98%) then Gini Index (F1 ≈97%)	Stable and balanced	More generalized configuration
20	Gain Ratio ≈ Information Gain ≈ Gini	Plateau	Overfitting avoided perhaps due to pruning

iii) Random Forest



A random forest is an ensemble machine learning algorithm that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting. It is particularly effective for high-dimensional datasets like our dataset.

In RapidMiner, the following key parameters were configured:

Random Forest

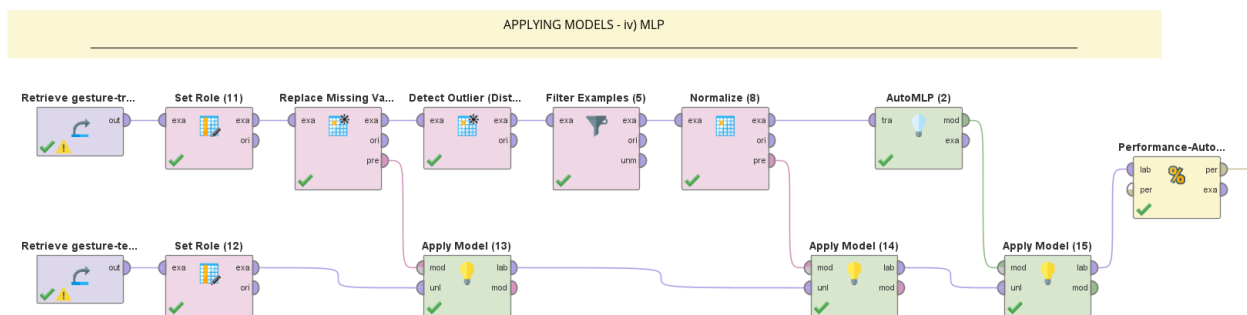
number of trees	300
criterion	gini index
maximal depth	25
☒ apply pruning	
74% of users kept 'no':	
yes	26%
no	74%
confidence	0.1
☒ apply prepruning	
minimal gain	0.01
minimal leaf size	2
minimal size for split	4
number of prepruning alternatives	3
☐ random splits	
☐ guess subset ratio	
subset ratio	0.2
voting strategy	confidence vote

Initially, it seemed strange that the single Decision Tree outperformed the Random Forest, since ensembles usually perform better. This prompted us to experiment with different parameters. By increasing the number of trees from 100 to 300, setting the maximum depth from 10 to 25, and using the Gini index criterion, the Random Forest learned more detailed patterns in the gestures while still focusing on the most informative features. Deeper trees captured subtle differences between similar gestures, and more trees stabilized predictions. These changes allowed the model to generalize better, resulting in a much higher F1-score of 93.0%.

The “guess subset ratio” parameter in RapidMiner adds extra randomness by automatically varying how many features are selected at each node of each tree. After turning this off, and applying pruning and pre pruning as well, we got the F1-score up to 99.11%. This shows how too much randomization in high dimensional data can hurt performance and how pruning can help improve precision and recall by making the model more simple.

We chose the Gini index for our Random Forest after observing that Gain Ratio performed poorly after PCA. Gain Ratio is sensitive to feature distributions, which changed after PCA, causing the F1-score to drop drastically. In contrast, Gini focuses on class purity and is less affected by transformed or compressed features. This made it much more robust when using PCA, leading to a large improvement in performance (F1-score increased from around 22% to 70%). Therefore, Gini was the better choice for handling reduced dimensionality gesture data.

iv) AutoMLP



The AutoMLP operator automatically searches for the best set of MLP models, which is crucial for handling the non-linear relationships often present in complex sensor data. The following key parameters were configured:

Parameters

AutoMLP (2) (AutoMLP)

training cycles10

number of generations10

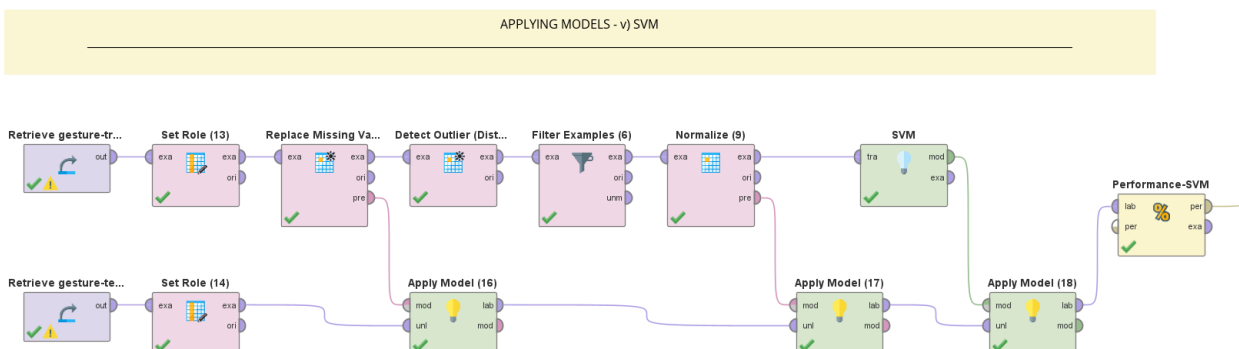
number of esemble mlps4

Training cycles is equal to 10, which means the number of learning process repetitions.

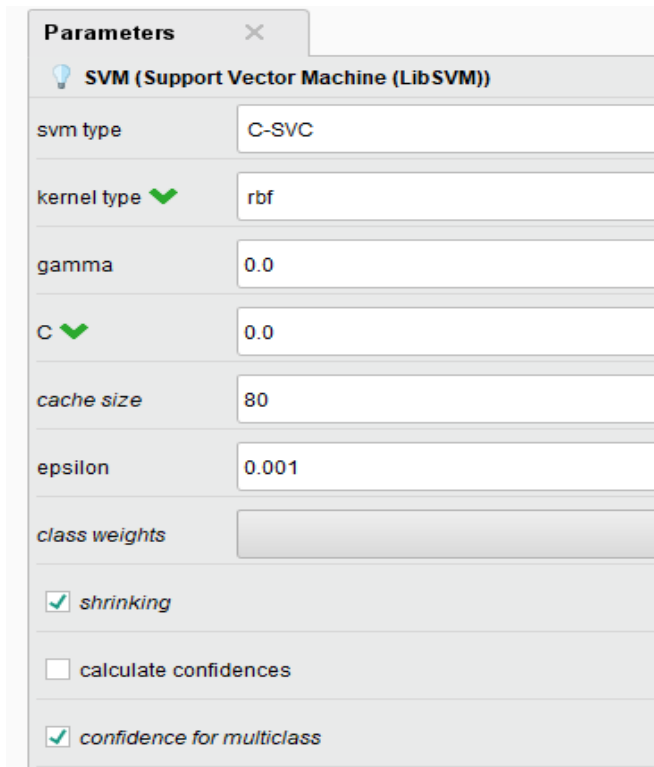
The number of generations is equal to 10, which means there will be 10 generations, and in each generation, a set of models is improved based on its performance

There are 4 ensemble MLPs to ensure that the final prediction is stabilized and further minimizes the risk of overfitting.

v) SVM



SVM finds the optimal hyperplane that maximizes the margin between classes, using kernel functions (RBF) to handle non-linear boundaries. The following key parameters were configured:



Parameters	
SVM (Support Vector Machine (LibSVM))	
svm type	C-SVC
kernel type ✓	rbf
gamma	0.0
C ✓	0.0
cache size	80
epsilon	0.001
class weights	
☒ shrinking	
☐ calculate confidences	
☒ confidence for multiclass	

RBf was chosen as Kernel Type, which is suitable for our Non-Linear dataset. Therefore, body movements and changes in velocity cannot be effectively separated by a simple, straight line in the original feature space.

Cache Size = 80, so SVM can save some values while training in the cache memory instead of calculating them again and again. As a result, it will increase the efficiency and speed of the training.

Epsilon is the value of Tolerance in the Optimization algorithm; we gave it a value of 0.001, so it stops once the accuracy optimization is below 0.001. As a result, we will have a balance between accuracy and time consumption.

At first, we set both C and Gamma to 0 in the SVM settings. After running the model, the F1 score turned out to be very low, around 67.1%, which was much worse than the other algorithms.

Later, we realized that having both values set to zero doesn't make sense mathematically. When $C = 0$, the model ignores all errors and basically doesn't learn. And when $\text{Gamma} = 0$, all the data points look the same, so the SVM can't tell the classes apart.

After we fixed this by changing C to 15 and Gamma to 0.001, the performance improved a lot; the F1 score went up to 85.84%, which is a big jump of 18.74%. We also tested the same setup with PCA, and the result was about 81.99%, which actually beat all the other models after PCA. Still, we didn't include it in the "Results with PCA" section since it wasn't one of the top two models before PCA.

Parameters ×

 **SVM (Support Vector Machine (LibSVM))**

svm type	C-SVC
kernel type 	rbf
gamma	0.001
C 	15.0
cache size	80
epsilon	0.001
class weights	
☒ <i>shrinking</i>	

2. Results

Looking at the results from all the above classifiers, we used the `weighted_mean_recall` and `weighted_mean_precision` to calculate the `f1_score`. In this case, since the classes are imbalanced, accuracy might not be the best measure.

In order to pick these criterions, we selected them as below, for all performance selections.

main criterion

weighted_mean_precision

☒ accuracy

☐ classification error 

☐ kappa

☒ weighted mean recall

☒ weighted mean precision

Precision measures how trustworthy the positive predictions are and generally a measure of reliability i.e., how confident we can be to predict the positive prediction.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall is the measure of sensitivity, i.e., how many actual positives were caught

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 score balances out the precision and recall and helpful when the requirement is to see how well the model handles the positive cases. To calculate the f1_score, we manually gathered the recall and precision to use the formula,

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Below shows the summary of all applied classifiers along with their respective f1_scores. As seen clearly, the two winners are **Random Forest (RF)** and **Decision Tree (DT)** with 99,1% and 96,72% respectively, which we took further for the second part of the task.

CLASSIFIER	wighted_mean_recall	wighted_mean_precision	calculated f1-score
kNN	59,26	64,82	61,91542876
DT	96,66	96,8	96,72994934
RF	99,22	99	99,10987791
MPL	86,4	87,97	87,17793198
SVM	85,43	86,25	85,83804171

3. Results with PCA

We applied Principal Component Analysis (PCA) in RapidMiner for dimensionality reduction, retaining 80% and 90% of the variance by combining features into principal components. Tree-based models (Decision Tree, Random Forest) performed poorly with PCA because it rotates the feature space and can discard low-variance but class-relevant information; trees rely on clean, original axes. In contrast, the MLP benefited: MLP with PCA at 90% achieved the best F1 (~0.78), which we recommend.

Parameters ✕

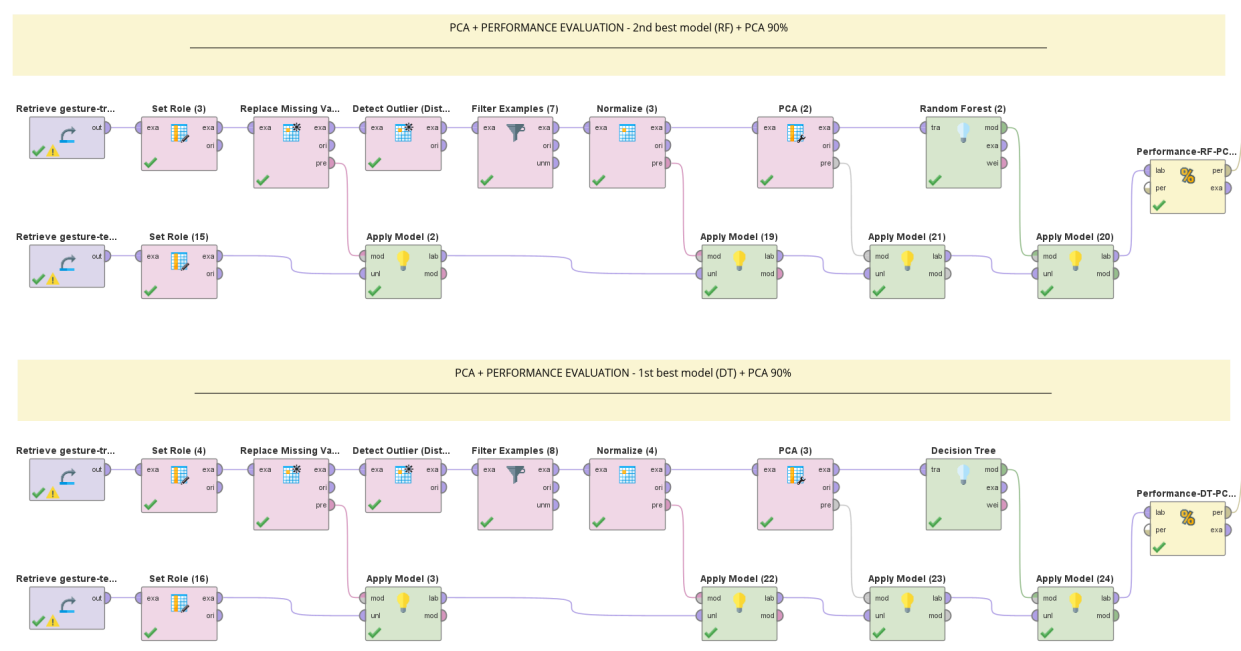
PCA (7) (Principal Component Analysis)

dimensionality reduction keep variance ⓘ

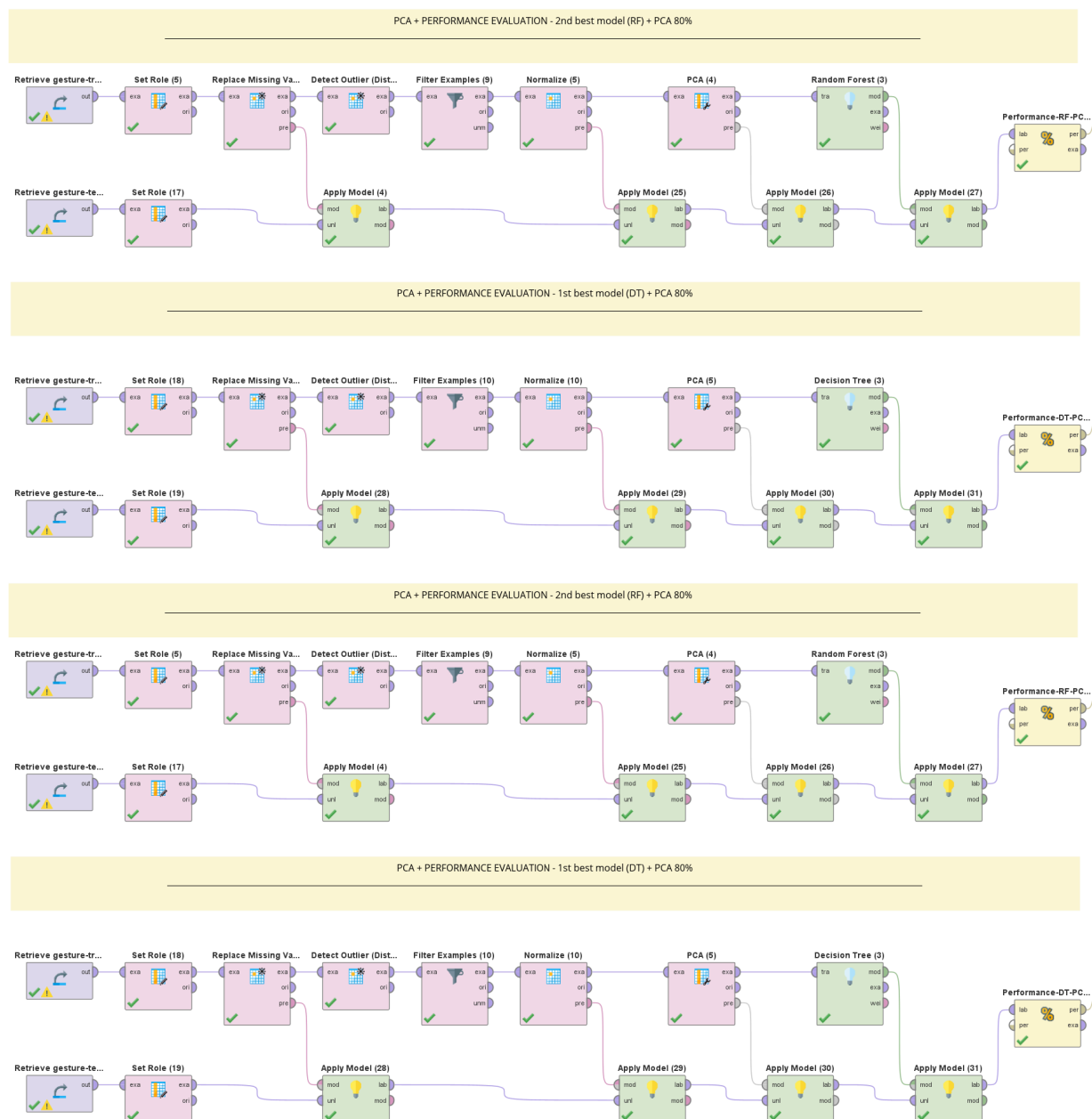
variance threshold 0.9 ⓘ

BEST 2 classifiers	wighted_mean_recall	wighted_mean_precision	calculated f1-score
RF-PCA-80	64,55	67,3	65,89632158
DT-PCA-80	19,85	23	21,3092182
RF-PCA-90	68,01	69,63	68,81046643
DT-PCA-90	18,35	23,05	20,43321256

90%



80%



4. Conclusion

In this task, we compared different classification algorithms for the gesture recognition data. Among all models tested, the Random Forest and Decision tree performed the best, achieving F1-scores of around 96% and 99% respectively.

We learned that the type of data really affects which classifiers work best. The gesture data is high-dimensional and has complex, non-linear patterns, as well as variations between performers. Random Forest and Decision tree handled this best. Random Forest uses many trees and looks at random subsets of features, which helps it avoid overfitting and capture complex patterns.

Models like k-NN and SVM did not perform as well because they struggle with many features and class imbalances. We were surprised that MLP did not do better, as a neural network, it can learn non-linear relationships which we thought would make it better at telling similar gestures apart.

PCA experiments showed that dimensionality reduction can help with speed but may also remove critical information and end up reducing precision and recall. Overall, the main lessons we learned were:

1. Understanding the data is essential to picking the right classifier
2. PCA is not always helpful
3. Tuning parameters is key to getting the best performance

CLASSIFIER	wighted_mean_recall	wighted_mean_precision	calculated f1-score
kNN	59,26	64,82	61,91542876
DT	96,66	96,8	96,72994934
RF	99,22	99	99,10987791
MPL	86,4	87,97	87,17793198
SVM	85,43	86,25	85,83804171
BEST 2 classifiers	wighted_mean_recall	wighted_mean_precision	calculated f1-score
RF-PCA-80	64,55	67,3	65,89632158
DT-PCA-80	19,85	23	21,3092182
RF-PCA-90	68,01	69,63	68,81046643
DT-PCA-90	18,35	23,05	20,43321256